

EXPERIMENT – 6

Name: Sam Fawaz Hadi AL-Basara

UID: 24BAI70236

Section/Group: 24AIT_NTPP 2 (A)

Branch/Semester: CSE_AIML (CS221) / 5th

Subject Code: 24 CSP-337

Subject Name: Full Stack - II

Date of Performance: 31/8/2026

Submitted to: Ms. Lovleen Kaur (E20454)

Experiment – 6.1

1. **AIM :** To implement paginated and sortable REST APIs for efficient and scalable data retrieval..

2. OBJECTIVE

- 1- To understand pagination and sorting concepts
- 2- To implement pageable APIs using Spring Data
- 3- To optimize data retrieval for large datasets
- 4- To improve API performance and usability

3. SOFTWARE REQUIREMENT

- 1- Java (JDK 17+)
- 2- Spring Boot
- 3- IDE (IntelliJ / Eclipse / VSCode)
- 4- Postman / API testing tool

4. THEORY:

In real-world applications, datasets can grow significantly large, making it inefficient to retrieve all records in a single API call. **Pagination** addresses this issue by dividing data into smaller chunks (pages), allowing clients to request only a subset of data at a time.

Sorting enhances usability by allowing data to be ordered based on specific fields such as creation date, popularity, or relevance.

Spring Boot provides built-in support for pagination and sorting through the **Pageable** interface, which simplifies API design and ensures efficient query execution.

Benefits of pagination and sorting include:

- Reduced server load
- Faster response times

- Improved user experience
- Better scalability

5. IMPLEMENTATION/PROCEDURE:

1. Create a Spring Boot Maven project.
2. Configure Java 25 and required dependencies.
3. Configure the H2 database.
4. Create the User entity with uid and name.
5. Create UserRepository by extending JpaRepository.
6. Create UserRequest for receiving user data.
7. Create PageResponse for paginated results.
8. Create UserService for user operations.
9. Implement pagination using Pageable.
10. Implement sorting using Sort.
11. Create UserController with REST endpoints.
12. Implement POST /api/users to add users.
13. Implement GET /api/users to retrieve paginated and sorted users.
14. Implement DELETE /api/users/{uid} to delete users.
15. Create the HTML, CSS, and JavaScript frontend.
16. Connect the frontend with REST APIs using JavaScript fetch().
17. Add pagination controls and sorting controls.
18. Run the application using Maven.
19. Open the application in a web browser.
20. Test adding, displaying, sorting, deleting, and navigating user records.

6. CODE

User.java

@Entity

@Table(name = "users")

public class User {

@Id

private String uid;



```
private String name;
```

```
public User() {}
```

```
public User(String uid, String name) {
```

```
    this.uid = uid;
```

```
    this.name = name;
```

```
}
```

```
public String getUid() {
```

```
    return uid;
```

```
}
```

```
public void setUid(String uid) {
```

```
    this.uid = uid;
```

```
}
```

```
public String getName() {
```

```
    return name;
```

```
}
```

```
public void setName(String name) {
```

```
    this.name = name;
```

```
}
```

```
}
```

UserRepository.java

```
public interface UserRepository  
    extends JpaRepository<User, String> {  
}
```

PageResponse.java

```
public class PageResponse<T> {
```

```
    private List<T> content;
```

```
    private int pageNumber;
```

```
    private int pageSize;
```

```
    private long totalElements;
```

```
    private int totalPages;
```

```
    private boolean first;
```

```
    private boolean last;
```

```
    // Constructors, getters and setters
```

```
}
```

UserService.java

```
@Service
```

```
public class UserService {
```

```
    private final UserRepository userRepository;
```

```
    public UserService(UserRepository userRepository) {
```

```
        this.userRepository = userRepository;
```

```
}
```

```
public User createUser(UserRequest request) {

    return userRepository.save(
        new User(
            request.getUid(),
            request.getName()
        )
    );
}

public PageResponse<User> getUsers(Pageable pageable) {

    Page<User> page =
        userRepository.findAll(pageable);

    return new PageResponse<>(
        page.getContent(),
        page.getNumber(),
        page.getSize(),
        page.getTotalElements(),
        page.getTotalPages(),
        page.isFirst(),
        page.isLast()
    );
}

public void deleteUser(String uid) {
    userRepository.deleteById(uid);
}
}
```

UserController.java

```
@RestController
@RequestMapping("/api/users")
@CrossOrigin(origins = "*")
public class UserController {
```

```
private final UserService userService;

public UserController(UserService userService) {
    this.userService = userService;
}

@PostMapping
public ResponseEntity<User> createUser(
    @Valid @RequestBody UserRequest request) {

    return ResponseEntity
        .status(HttpStatus.CREATED)
        .body(userService.createUser(request));
}

@GetMapping
public ResponseEntity<PageResponse<User>> getUsers(
    @PageableDefault(
        page = 0,
        size = 5,
        sort = "uid",
        direction = Sort.Direction.ASC
    )
    Pageable pageable) {

    return ResponseEntity.ok(
        userService.getUsers(pageable)
    );
}

@DeleteMapping("/{uid}")
public ResponseEntity<Void> deleteUser(
    @PathVariable String uid) {

    userService.deleteUser(uid);

    return ResponseEntity.noContent().build();
}
```

```
}  
}
```

script.js

```
const API_URL =
```

```
"http://localhost:8080/api/users";
```

```
let currentPage = 0;
```

```
async function loadUsers(page) {
```

```
    currentPage = page;
```

```
    const pageSize =
```

```
        document.getElementById("pageSize").value;
```

```
    const sortField =
```

```
        document.getElementById("sortField").value;
```

```
    const sortDirection =
```

```
        document.getElementById("sortDirection").value;
```

```
    const url =
```

```
        `${API_URL}?page=${currentPage}` +
```

```
        `&size=${pageSize}` +
```

```
        `&sort=${sortField},${sortDirection}`;
```

```
const response = await fetch(url);

const result = await response.json();

displayUsers(result);
}
```

Pagination controls:

```
function previousPage() {

  if (currentPage > 0) {
    loadUsers(currentPage - 1);
  }
}
```

```
function nextPage() {

  loadUsers(currentPage + 1);
}
```

Adding a user:

```
async function addUser(uid, name) {

  await fetch(API_URL, {

    method: "POST",

    headers: {
      "Content-Type": "application/json"
    },
```

```
body: JSON.stringify({  
  uid: uid,  
  name: name  
})  
});
```

```
loadUsers(0);  
}
```

Deleting a user:

```
async function deleteUser(uid) {
```

```
  await fetch(  
    `${API_URL}/${uid}`,  
    {  
      method: "DELETE"  
    }  
  );
```

```
  loadUsers(currentPage);  
}
```

Sam 24BAI70236

7. OUTPUT

Read less. *Understand more.*

A working field guide to pageable REST APIs. Tune the request, watch the payload stay lean, and see how Spring Data handles the heavy lifting.

SF Sam Fawaz Hadi AL-Basara
UID-248A170236 • Full Stack - II

DATASET
12
ARTICLES IN CATALOGUE
IN-MEMORY H2 DATABASE

LIVE EXPLORER

Browse the knowledge stream

Pagination and sorting are applied by the REST API.

GET /api/articles

SHOW 6 PER PAGE SORT BY Oldest first

Reset view

Showing 1-6 of 12 articles

32 ms response

ARCHITECTURE REST endpoints as small contracts Leo Chen · 5 min Published Nov 29, 2024	PERFORMANCE Streams, pages, and the network Sana Rahman · 7 min Published Dec 7, 2024 3.4k reads	ARCHITECTURE Measuring the cost of a request Devon Lee · 8 min Published Dec 15, 2024 2.9k reads
SPRING BOOT Readable errors, happier clients Nora Williams · 6 min Published Dec 22, 2024 1.8k reads	DATA H2 to production: what changes Arjun Mehta · 11 min Published Jan 3, 2025 2.3k reads	API DESIGN The shape of a useful response Maya Patel · 4 min Published Jan 9, 2025 1.5k reads

Fig 6.1-Final User Management Dashboard Showing Pagination and Sorting REST API

8. LEARNING OUTCOME

- · Explain the importance of pagination and sorting in scalable REST APIs.
- · Implement pagination using Spring Data JPA Pageable.
- · Implement dynamic sorting using the Sort interface.
- · Develop REST APIs for efficient database operations.
- · Integrate a frontend with a Spring Boot backend.
- · Test and verify paginated and sortable API responses.
- · Understand how pagination improves performance and usability when handling large datasets.

Sam 24BAI70236

Experiment – 6.2

1. **AIM:**To optimize backend read operations using caching and efficient database query strategies..

2. OBJECTIVE

- 1- To identify and resolve inefficient database queries
- 2- To understand the N+1 query problem
- 3- To implement caching mechanisms
- 4- To improve API performance through optimization

3. SOFTWARE REQUIREMENT

- 1- Spring Boot
- 2- Database (MySQL/PostgreSQL)
- 3- Ehcache (or similar caching tool)
- 4- Apache JMeter (for performance testing)

4. THEORY

Backend performance is critical in applications that handle large volumes of read requests. Inefficient queries and repeated database access can significantly degrade performance.

One common issue is the **N+1 query problem**, where fetching related entities results in multiple database queries instead of a single optimized query. This leads to increased latency and unnecessary database load.

This problem can be resolved using techniques such as **JOIN FETCH**, which retrieves related data in a single query, reducing database round trips.

Another key optimization technique is **caching**, where frequently accessed data is stored in memory to avoid repeated database queries. Tools like Ehcache enable efficient caching strategies, improving response times and reducing system load.

For complex queries, **native SQL queries** can be used to gain better control over execution and performance.

Finally, **performance benchmarking** using tools like Apache JMeter helps evaluate system performance under load and validate the effectiveness of optimization strategies.

5. IMPLEMENTATION/PROCEDURE:

1. Create a Spring Boot Maven project named **Experiment6.2**.
2. Configure Java 25 and required Spring Boot dependencies.
3. Configure the H2 in-memory database in application.properties.
4. Create the User and Address entity classes.
5. Establish a one-to-one relationship between User and Address.
6. Create UserRequest for validated user registration.
7. Create UserRepository with normal JPQL, JOIN FETCH, native SQL, and sorting queries.
8. Enable caching using @EnableCaching.
9. Implement service methods for normal, optimized, cached, native, and sorting queries.
10. Create REST API endpoints in UserController.
11. Develop the HTML, CSS, and JavaScript frontend.
12. Run the Spring Boot application.
13. Register users through the web interface.
14. Execute the normal query to demonstrate the N+1 problem.
15. Execute the optimized JOIN FETCH query.
16. Execute the cached query and observe repeated-request performance.
17. Execute the native SQL and sorting queries.
18. Use Apache JMeter with **10 threads and 5 loops** to test the endpoints.
19. Compare response times and observe the performance improvement.

CODE

Main Application

@SpringBootApplication

```
public class Experiment62Application {
```

```
    public static void main(String[] args) {
```

```
        SpringApplication.run(Experiment62Application.class, args);
```

```
    }
```

```
}
```

Enable Caching

@Configuration

@EnableCaching

```
public class CacheConfig {  
  
}
```

Repository – Main Query Methods

```
public interface UserRepository extends JpaRepository<User, Long> {  
  
    @Query("SELECT u FROM User u")  
  
    List<User> findAllNormal();  
  
    @Query("SELECT DISTINCT u FROM User u LEFT JOIN FETCH u.address")  
  
    List<User> findAllWithAddress();  
  
    @Query(value = "SELECT * FROM users", nativeQuery = true)  
  
    List<User> findAllNative();  
  
  
    List<User> findAllByIdAsc();  
    List<User> findAllByNameAsc();  
}
```

Service – Query Optimization and Caching

```
@Service  
  
public class UserService {  
  
    private final UserRepository userRepository;  
  
    public UserService(UserRepository userRepository) {  
  
        this.userRepository = userRepository;  
  
    }  
  
    public List<User> getUsersNormal() {  
  
        List<User> users = userRepository.findAllNormal();  
  
        users.forEach(user -> {
```

```
        if (user.getAddress() != null) {

            user.getAddress().getCity();

        }

    });

    return users;

}

public List<User> getUsersOptimized() {

    return userRepository.findAllWithAddress();

}

@Cacheable("users")

public List<User> getUsersCached() {

    return userRepository.findAllWithAddress();

}

public List<User> getUsersNative() {

    return userRepository.findAllNative();

}

}
```

Controller – Main Endpoints

```
@RestController

@RequestMapping("/api/users")

public class UserController {

    private final UserService userService;

    @GetMapping("/normal")

    public List<User> normal() {

        return userService.getUsersNormal();

    }

}
```

```
}

@GetMapping("/optimized")

public List<User> optimized() {

    return userService getUsersOptimized();

}

@GetMapping("/cached")

public List<User> cached() {

    return userService getUsersCached();

}

@GetMapping("/native")

public List<User> nativeQuery() {

    return userService getUsersNative();

}

}
```

Frontend – API Request

```
async function loadUsers(type) {

    const response = await fetch(`/api/users/${type}`);

    const users = await response.json();

    displayUsers(users);

}
```

6. OUTPUT

- Native SQL execution
- Response-time benchmarking

Add User

Query optimization tests

Sort users

Response time: 12 ms

User directory

ID	UID	NAME	CITY	COUNTRY	ACTION
2	E20455	Amit Sharma	Delhi	India	Delete
3	E20456	Priya Singh	Mumbai	India	Delete
4	E20457	Rahul Kumar	Chandigarh	India	Delete
5	24BAI70236	Sam Fawaz Hadi AL-Basara	Bengaluru	India	Delete
6	TEST001	Sample User	Dubai	UAE	Delete

Fig. 6.2 –Final Caching and Query Optimization Application with Query Performance Results

7. LEARNING OUTCOME

- · Understand the **N+1 query problem** in Spring Data JPA.
- · Use JOIN FETCH to optimize database queries.
- · Implement **caching using Spring Cache and Ehcache**.
- · Perform and compare **JPQL and native SQL queries**.
- · Apply **sorting operations** to database records.
- · Measure and compare API performance using **Apache JMeter**.
- · Understand how **query optimization and caching improve application performance** by reducing unnecessary database operations.

Sam 24BAI70236